Лабораторна робота № 7

Тема. Робота з файлами та форматом JSON.

Мета: отримати практичні навички роботи з файлами та навчитися виконувати операції збереження, завантаження й обробку даних у форматі JSON.

Короткі теоретичні відомості

pathlib – це стандартний модуль для роботи з шляхами до файлів та папок. *Path* – це головний клас, який представляє шлях. Для об'єднання частин шляху використовується оператор /. Приклад:

```
from pathlib import Path

config_path = Path("config.ini")

# поточний робочий каталог
current_dir = Path.cwd()

full_path = Path("img") / "1.jpeg"

print(full_path.name) # 1.jpeg
print(full_path.stem) # 1
print(full_path.suffix) # .jpeg
print(full_path.parent) # img
print(full_path.exists()) # True
print(full_path.is_file()) # True
```

Також *pathlib* надає методи для читання та запису файлів без необхідності окремо використовувати *open()*. Режими відкриття (*mode*): *'r'* – читання (за замовчування); *'w'* – запис (перезаписує файл); *'a'* – дозаписує; *'x'* – ексклюзивне створення (помилка, якщо файл існує); *'b'* – бінарний режим (додається до інших, наприклад *'rb'*); *'t'* – текстовий режим (за замовчуванням). Приклад читання та запису файлу:

```
from pathlib import Path

file_path = Path("example.txt")

with file_path.open("w") as file:
    file.write("Довгий текст...")
```

```
with file_path.open("r") as file:
    text = file.read()

print(text)

# Зручні методи
file_path.write_text("Короткий текст") # запис тексту (перезаписує
файл)
text = file_path.read_text() # читання всього тексту
lines = file_path.read_text().splitlines() # читання рядків у список
data = file_path.read_bytes() # читання вмісту файлу у вигляді байтів
```

JSON (JavaScript Object Notation) – це легкий текстовий формат обміну даними, заснований на синтаксисі об'єктів JavaScript. Приклад типових JSON-даних:

```
{
    "name": "Олена",
    "age": 18,
    "is_student": true,
    "courses": ["Вища математика", "Дискретна математика"],
    "address": {
        "city": "Кременчук",
        "street": "Університетська"
    }
}
```

Серіалізація – це процес перетворення об'єкта або структури даних у формат, придатний для зберігання або передачі (наприклад, JSON, XML, байти).

Десеріалізація – це процес відновлення об'єкта або структури даних із серіалізованого формату назад у початковий вигляд.

Для роботи з JSON існує вбудований модуль *json*. Приклад:

```
import json
from pathlib import Path

user = {"name": "Олена", "age": 18, "active": True}
```

```
json_str = json.dumps(user, ensure_ascii=False, indent=2) #
серіалізація
print(json_str)

user = json.loads(json_str) # десеріалізація
print(user)

file_path = Path("config.json")
file_path.write_text(json.dumps(user,
                                ensure_ascii=False,
                                indent=4), encoding="utf-8")

if file_path.exists():
    print(json.loads(file_path.read_text(encoding="utf-8"))["name"])
```

Порядок виконання роботи

1. Уважно вивчити короткі теоретичні відомості та літературу.

2. Згідно з варіантом виконати серіалізацію словника та записати його в файл *output.json*. Потім зчитати дані з цього файлу, провести десеріалізацію та виконати обробку.

3. Написати звіт з виконання лабораторної роботи.

Варіанти завдань

№ варіанта	Словник	Обробка після десеріалізації
1	`{ "id": 1, "numbers": [10, 20, 30, 40, 50], "active": True }`	Обчислити суму всіх чисел за ключем *"numbers"* та вивести результат.
2	`{ "id": 2, "numbers": [5, 15, 25, 35, 45], "name": "Варіант 2" }`	Обчислити середнє арифметичне списку *"numbers"* та вивести його з двома знаками після коми.

3	<pre>{ "id": 3, "numbers": [12, 7, 9, 24, 31], "desc": "Варіант 3" }</pre>	Вивести в консоль лише парні числа зі списку *"numbers"*.
4	<pre>{ "id": 4, "numbers": [12, 3, 15, 27, 41], "meta": {"type": "text"} }</pre>	Знайти максимальне та мінімальне значення у списку *"numbers"* і вивести їх.
5	<pre>{ "id": 5, "unsorted": [42, 15, 20, 99, 11], "flag": False }</pre>	Відсортувати список *"unsorted"* за зростанням та вивести оновлений список.
6	<pre>{ "id": 6, "title": "Серіалізація", "options": [1, 2, 3] }</pre>	Перетворити значення *"title"* у верхній регістр та зберегти результат у *"title_upper"*.
7	<pre>{ "id": 10, "factors": [42, 2, 45, 99, 4], "name": "Варіант 7" }</pre>	Обчислити добуток всіх чисел зі списку *"factors"* та вивести результат.

Контрольні питання

1. Який модуль використовується для роботи з форматом JSON?
2. Як називається процес перетворення об’єкта у рядок JSON?
3. Як називається зворотний процес – перетворення рядка JSON на об’єкт?

4. Який метод об’єкта *Path* дозволяє перевірити, чи існує файл або каталог за цим шляхом?

5. Який метод об’єкта *Path* використовується для читання всього тексту з файлу за один виклик?